

Parallel Prime Number Computation using Multi-Process Method (FASTprime)

Name: Anudeep Devineni

Subject: Operating Systems Lab

Assignment: Group Assignment – FASTprime

1. Aim

To find prime numbers in a given range using multiple child processes and measure execution time using parallel processing.

2. Objective

- To understand how parallel processing works.
- To use `fork()` for creating processes.
- To divide work among processes.
- To reduce execution time.
- To compare performance using graphs.

3. Tools Used

- **Language:** C Programming Language
- **OS:** Linux Operating System
- **Compiler:** GCC Compiler
- **System Calls:** `fork()`, `signal()`, `kill()`, `open()`, `write()`, `close()`, `gettimeofday()`, `system("lscpu")`

4. System Configuration

(Replace the example below with your actual system details)

- **Processor:** Intel i7
- **Logical Cores:** 12
- **RAM:** 16 GB
- **OS:** Ubuntu Linux

5. Program Description

In this program, prime numbers are calculated in parallel using multiple child processes.

1. First, the CPU information is displayed using the `lscpu` command.
2. The user enters the range and number of processes.
3. The range is divided equally among all child processes. Each child process checks prime numbers in its assigned sub-range.

4. All prime numbers found are written into a file named `prime.txt`.
5. Execution time is measured before and after computation using the system clock.

6. Algorithm

1. Start the program.
2. Display CPU information.
3. Read lower and upper range.
4. Read number of processes.
5. Open output file.
6. Start timer.
7. Divide range equally.
8. Create child processes using `fork()`.
9. Each child computes primes.
10. Write primes to file.
11. Child sends signal to parent.
12. Parent waits for all children.
13. Stop timer.
14. Calculate execution time.
15. Display result.
16. End program.

7. Code

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>

#include <fcntl.h>

#include <string.h>

#include <signal.h>

#include <sys/time.h>


// function to check prime

int isPrime(int n)

{

    if(n < 2)

        return 0;
```

```

    for(int i = 2; i <= n/2; i++)
    {
        if(n % i == 0)
            return 0;
    }

    return 1;
}

// signal handler
void done(int sig)
{
    printf("Child completed its work\n");
}

int main()
{
    int rl, rh;
    int n;

    printf("CPU Details:\n");
    system("lscpu");

    // input range
    printf("\nEnter range (rl rh): \n ");

```

```

scanf("%d %d", &rl, &rh);

// number of children
printf("Enter number of child processes: ");
scanf("%d", &n);

// open file
int fd = open("prime.txt",
              O_CREAT | O_WRONLY | O_TRUNC,
              0644);

// register signal
signal(SIGUSR1, done);

struct timeval start, end;
gettimeofday(&start, NULL);

int size = (rh - rl + 1) / n;

for(int i = 0; i < n; i++)
{
    pid_t pid = fork();

    if(pid == 0)    // child
    {
        int s = rl + i * size;
        int e;

```

```

        if(i == n-1)
            e = rh;
        else
            e = s + size - 1;

        // find primes
        for(int j = s; j <= e; j++)
        {
            if(isPrime(j))
            {
                char buf[50];
                sprintf(buf, "%d\n", j);
                write(fd, buf, strlen(buf));
            }
        }

        // inform parent
        kill(getppid(), SIGUSR1);

        close(fd);
        exit(0);
    }
}

// parent waits
for(int i = 0; i < n; i++)

```

```
        wait(NULL);

gettimeofday(&end, NULL);

double t =

    (end.tv_sec - start.tv_sec) +

    (end.tv_usec - start.tv_usec)/1e6;

printf("\nExecution Time = %f sec\n", t);

close(fd);

}
```

8. Experimental Setup

The program was executed for different numbers of processes to observe performance improvement.

- **Processes tested:** 1,2,3,4,5,6,7,8,9,10,11,12

9. Input Ranges Tested

- 1000 – 10000
- 50000 – 100000
- 1000 – 100000

10. Result Tables

Range: 1000 - 10000 :

Child Processes	Execution Time (sec)
1	0.006833
2	0.005129
3	0.002783
4	0.003198
5	0.001952
6	0.002075
7	0.002286
8	0.001888
9	0.001746
10	0.001934
11	0.001961
12	0.001911

Range: 50000 - 100000 :

Child Processes	Execution Time (sec)
1	0.20503
2	0.143812
3	0.094182
4	0.082542
5	0.071969
6	0.059135
7	0.061952
8	0.050999
9	0.052583
10	0.052694
11	0.051911
12	0.046143

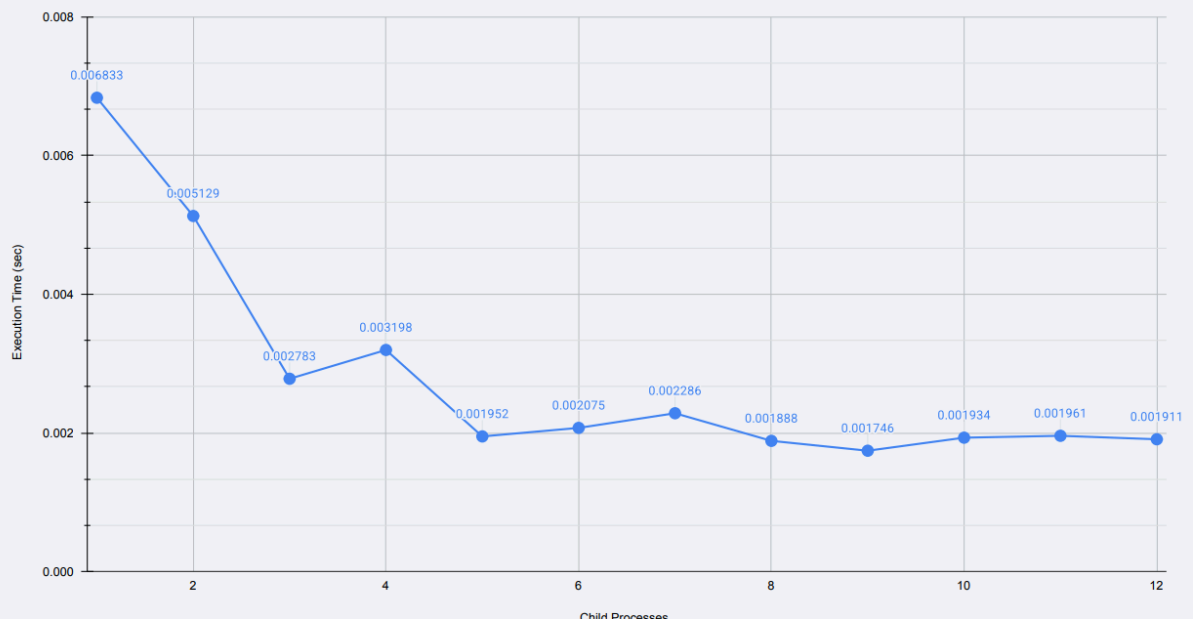
Range: 1000 - 100000 :

Child Processes	Execution Time (sec)
1	0.286105
2	0.208445
3	0.166095
4	0.123466
5	0.120231
6	0.090968
7	0.106869
8	0.105678
9	0.101888
10	0.082744
11	0.075462
12	0.072458

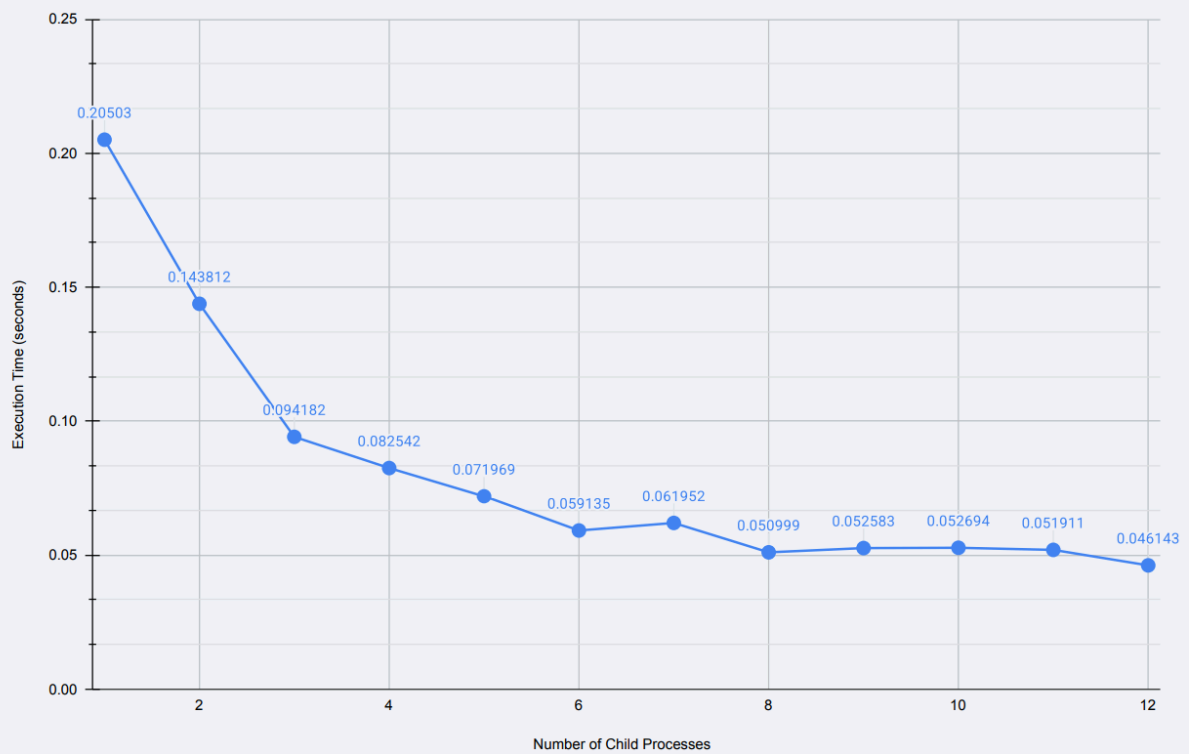
11. Graphs

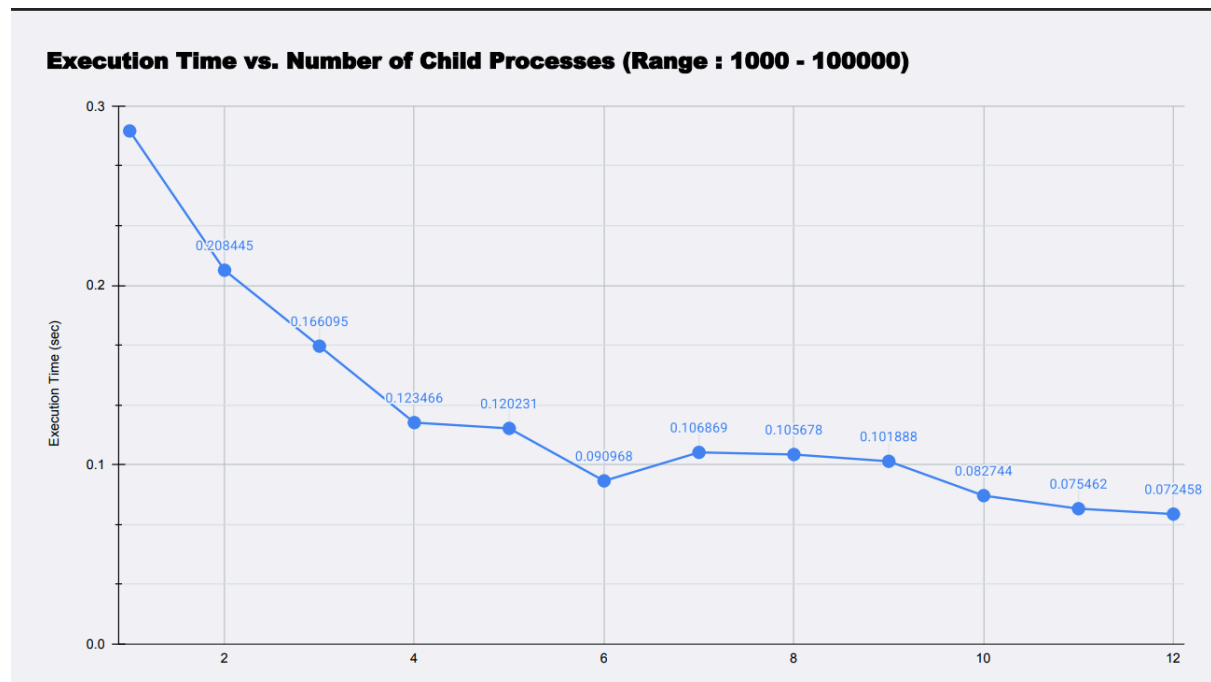
Graphs were plotted between **Number of Processes vs. Execution Time**. Separate graphs were created for each input range

Execution Time vs. Number of Child Processes (Range : 1000 - 10000)



Execution Time vs. Number of Child Processes (Range : 50000 - 100000)





12. Observations

- Execution time decreased as the number of processes increased.
- Parallel execution improved performance.
- Best performance was observed near the number of CPU cores.
- Increasing processes beyond core count increased overhead.
- Larger ranges showed more benefit from parallel processing.

13. Conclusion

From this experiment, it is clear that parallel processing reduces execution time significantly. By dividing work among multiple child processes, CPU utilization improves. However, creating too many processes can increase overhead and reduce efficiency.